

Отчёт о реализации функционала пользователей

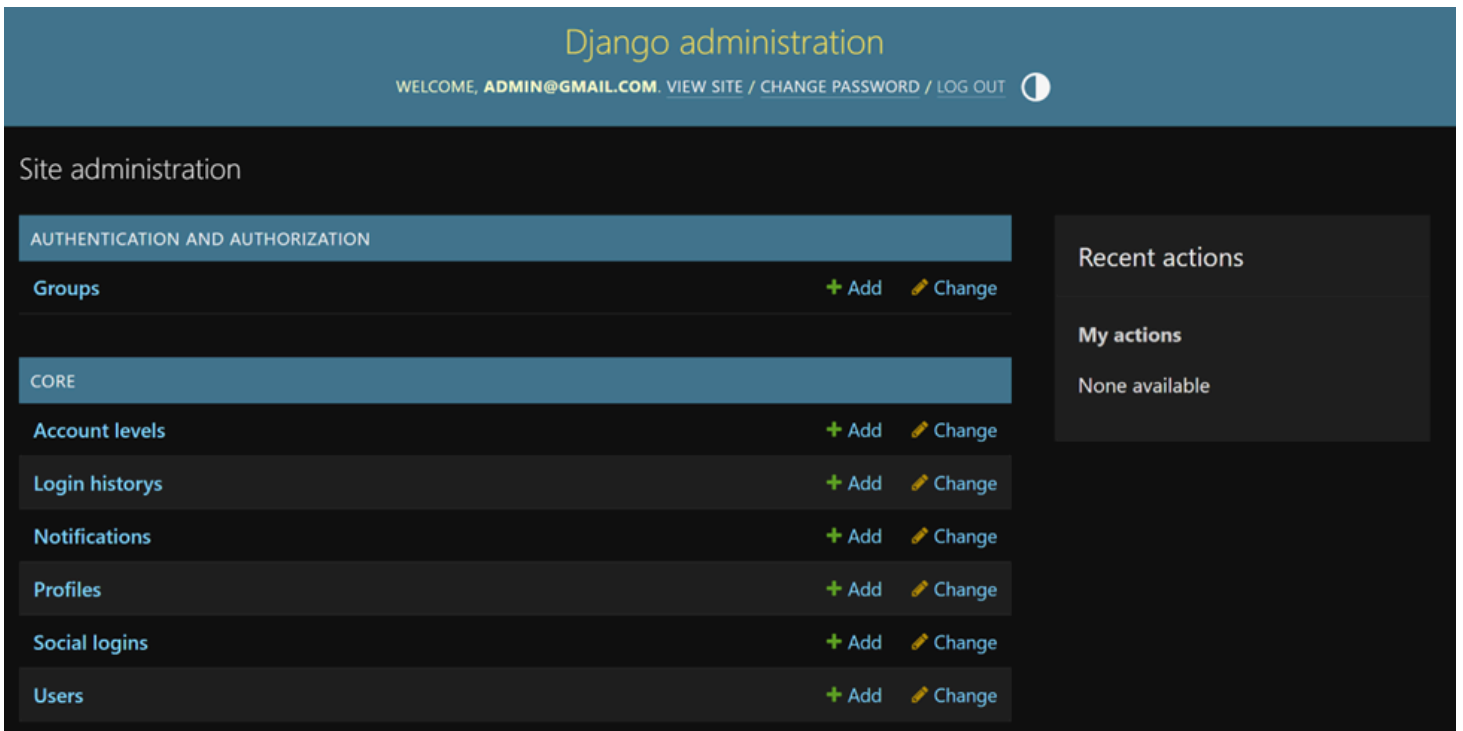
Админка и тестирование через admin.py

Мы подключили модели к Django Admin для удобного тестирования и управления:

```
from django.contrib import admin
from .models import User, Profile, AccountLevel, Notification, SocialLogin, LoginHistory
from django.contrib.auth.admin import UserAdmin as BaseUserAdmin

@admin.register(User)
class CustomUserAdmin(BaseUserAdmin):
    ordering = ('email',)
    list_display = ('email', 'is_staff', 'is_active', 'email_verified')
    fieldsets = (
        (None, {'fields': ('email', 'password')}),
        ('Permissions', {'fields': ('is_active', 'is_staff', 'is_superuser', 'groups', 'user_permissions')}),
        ('Important dates', {'fields': ('last_login', 'created_at')}),
    )
    add_fieldsets = (
        (None, {
            'classes': ('wide',),
            'fields': ('email', 'password1', 'password2', 'is_staff', 'is_superuser'),
        }),
    )

admin.site.register(Profile)
admin.site.register(AccountLevel)
admin.site.register(Notification)
admin.site.register(SocialLogin)
admin.site.register(LoginHistory)
```



Суперпользователь для доступа к админке:

URL: <https://voltusv.pythonanywhere.com/admin/>

Логин: admin@gmail.com

Пароль: admin123@gmail.com

URLs и маршрутизация

backend/users/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("register/", views.register_view, name="register"),
    path("login/", views.login_view, name="login"),
]
```

backend/allways_project/urls.py

```
from django.contrib import admin
from django.urls import path, include, re_path
from core.views import FrontendAppView

urlpatterns = [
    path('admin/', admin.site.urls),
    path("users/", include("users.urls")),
    re_path(r'^(?!admin|users/).*$', FrontendAppView.as_view(), name='home'),
]
```

Фронтенд

Шаблоны: register.html и login.html (backend/users/templates/users/)

Используется Bootstrap для стилизации и клиентской валидации

Email и пароль проверяются на клиенте, POST-запрос отправляется на сервер

URL:

Регистрация: <https://voltage.pythonanywhere.com/users/register/>

Вход: <https://voltage.pythonanywhere.com/users/login/>

Вход

Email

admin@gmail.com

Пароль

.....

Войти

Нет аккаунта? [Зарегистрироваться](#)

Регистрация

Email

admin@gmail.com

Пароль

.....

Повтор пароля

Зарегистрироваться

Уже есть аккаунт? [Войти](#)

Home > Core > Users

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

CORE

Account levels + Add

Login historys + Add

Notifications + Add

Profiles + Add

Social logins + Add

Users + Add

Select user to change

ADD USER +



Search

Action: ----- Go 0 of 4 selected

☐	EMAIL	IS STAFF	IS ACTIVE	EMAIL VERIFIED
☐	admin@gmail.com	✓	✓	✗
☐	test_user1@gmail.com	✗	✓	✗
☐	user1@example.com	✗	✓	✗
☐	user2@example.com	✗	✓	✗

4 users

FILTER

👁 Show counts

↓ By is staff

All
Yes
No

↓ By superuser status

All
Yes
No

↓ By is active

All
Yes
No

Views (backend/users/views.py)

```
from django.shortcuts import render, redirect
from django.contrib.auth import get_user_model, authenticate, login

User = get_user_model()

def register_view(request):
    if request.method == "POST":
        email = request.POST.get("email")
        password = request.POST.get("password")
        if User.objects.filter(email=email).exists():
            return render(request, "users/register.html", {"error": "Email занят"})
        user = User.objects.create_user(email=email, password=password)
        login(request, user)
        return redirect("/")
    return render(request, "users/register.html")

def login_view(request):
    if request.method == "POST":
        email = request.POST.get("email")
        password = request.POST.get("password")
        user = authenticate(request, email=email, password=password)
        if user:
            login(request, user)
            return redirect("/")
        return render(request, "users/login.html", {"error": "Неверный email или пароль"})
    return render(request, "users/login.html")
```

Модель пользователя (core/models.py)

Кастомная модель User на базе AbstractBaseUser + PermissionsMixin

Менеджер UserManager с методами create_user и create_superuser

Поля: email (уникальный), пароль, is_active, is_staff, email_verified, last_login, created_at, updated_at, failed_attempts

Примеры тестирования

Суперпользователь

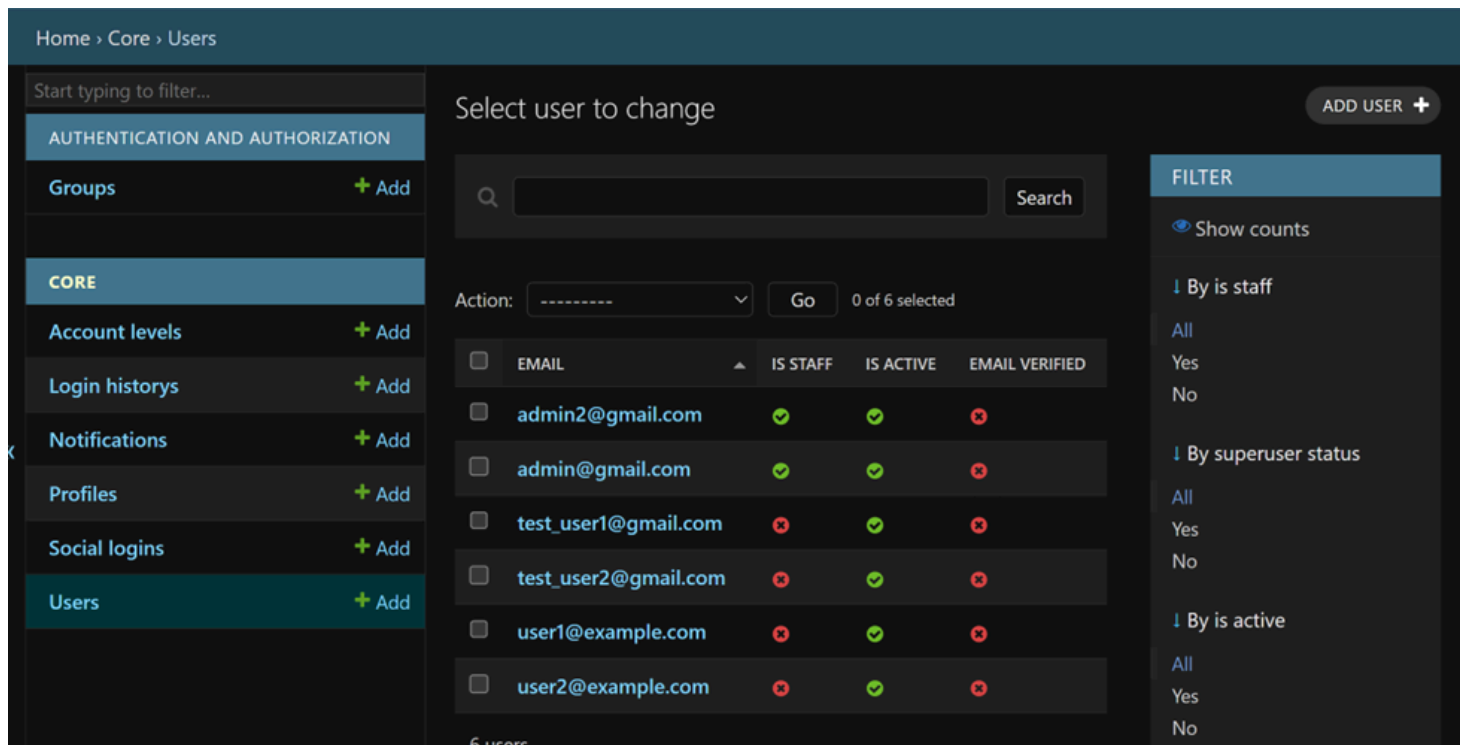
```
admin = User.objects.create_superuser("admin2@gmail.com", "adminpass")
print(admin) # admin2@gmail.com
```

Обычный пользователь

```
from django.contrib.auth import authenticate
```

```
user = authenticate(email="test_user2@gmail.com", password="12345")
print(user) # <User: test_user2@gmail.com>
```

```
user = authenticate(email="test_user2@gmail.com", password="wrongpass")
print(user) # None
```



Кратко как работает

Фронтенд: форма HTML + Bootstrap → POST-запрос на сервер

View: получает данные, создаёт пользователя (create_user) или проверяет (authenticate)

Модель: хэширует пароль и сохраняет в БД

База данных: хранит пользователей и связанные модели, обеспечивает проверку при авторизации

Примечания:

*** Убрал password_hash из [models.py](#), чтобы убрать ошибку, связанную с тем, что не получается авторизовать пользователя в системе